Online Payload Estimation Nearly Closes the Centered Case but Not the Off-Center One in Docked Modular Multirotors

Hao-Ting Ho
Department of Electrical Engineering, Tunghai University, Taichung, Taiwan
haoting.ho@outlook.com

***Abstract*—Several small quadrotor modules docked into one rigid body can be flown by a single centralized controller, but an unknown payload (mass and center-of-mass offset) invalidates the model that the controller's thrust allocation relies on. We study this problem in a batched rigid-body simulator. We build a ladder of classical controllers (QP thrust allocation, a yaw-rate limit, and recursive-least-squares payload estimation) and use a from-scratch PPO as a probe. The pre-registration was frozen after the classical exploration runs and defines three disjoint seed sets; the classical results were then recomputed on the confirmatory set (the off-center scores had been seen before the freeze). On exploration seeds, a 15% payload costs the best classical baseline in that condition 14.4% of its own nominal score, against 1.7% for $\pm 20\%$ thrust-coefficient error and 0.3% for docking error. On confirmatory seeds with a centered payload, the estimator-equipped stack keeps 0.69–0.82 of its nominal score on the tracking task and 0.95–0.99 on the position step. Its paired difference to a payload-informed reference is at most 0.007 (upper CI bound 0.019), within the point-mass payload model. For an off-center payload, a gap of 8/11/22% (6/7/8 modules) remains in the position-step task. A from-scratch PPO did not meet the pre-registered precondition for opening the main RL experiment within the recipe budget, and an exploratory residual PPO on top of the estimator stack did not close the remaining gap for two training seeds. These RL statements hold for these recipes and this compute budget only.**

***Index Terms*—modular aerial robots, payload estimation, thrust allocation, reinforcement learning, pre-registration**

I. Introduction

Small quadrotors can dock into a larger vehicle that flies as one rigid body [1]–[3]. Once docked, the thrust that each rotor must produce depends on the mass, inertia, and center of mass of the whole body. A payload of unknown mass and position changes all three.

We study this problem for a single centralized controller. We make no claim about decentralized control. Three questions drive the paper. (Q1) Which model error costs the most: payload, thrust-coefficient error, or docking error? (Q2) How much of the payload cost can a classical stack that estimates the payload online recover, within the point-mass payload model? (Q3) Can a learned controller whose actor cannot observe the payload close the gap that remains?

The main evidence is classical. Reinforcement learning (RL) enters as a probe with a pre-registered stopping rule. The main RL experiment would open only if a from-scratch policy on the six-module line reached 90% of the estimator stack's tracking score under a 15% payload. Development policies stayed below 90% of B1 on at least one task even without a payload, so that test was never run. The only RL result with a payload is an exploratory residual policy, and the negative outcome is reported as such. The study was pre-registered (version 1.0, tag `prereg-v1.0`) with three disjoint seed sets, and the classical results were recomputed on fresh confirmatory seeds after the freeze. Contributions:

1) Classical result: on exploration seeds, payload is the costliest of the three model errors we compared; on confirmatory seeds, the estimator stack stays close to the informed reference for a centered payload and leaves a gap that grows with module count for an off-center payload (Sections IV and VII).
2) RL result: the precondition of the pre-registered stopping rule was not met, so the main RL experiment was not opened, and an exploratory residual policy did not close the gap (Sections V and VII).
3) A protocol record: what was frozen, what was chosen after seeing classical results, what was never run, and one erratum (Section VI).

Non-claims: there is no hardware experiment; the payload is a point mass; the E3 condition lies at a corner of the training range and is not out-of-distribution; and no test of "RL versus classical" was run. Code and logs: https://github.com/Jadiouo/docked-multirotor-payload.

II. Related Work

Prior work on modular vehicles covers assembly, control, and configuration estimation. ModQuad demonstrated quadrotor modules that self-assemble in flight [1]. Later work extends the control of docked modules [2], adds model-predictive control with a thrust allocator [4], and plans self-reconfiguration without learning [5]. UFO estimates the configuration of a modular multirotor from several IMUs and refines it with an adaptive geometric controller [3]. In our reading of that paper, it needs an identification phase with hand rotation and assumes parallel thrust directions and a plate payload. Our estimator runs online from the first step with no identification phase, but it assumes a point-mass payload.

The closest learning work conditions one policy on the allocation matrix for fixed-motor-count families, whereas we study a docked rigid body with a payload and compare against

classical estimation. That work trains one PPO policy with recurrent memory for quadrotor and hexarotor families [6]. Other work adapts one policy across quadrotor scales or morphologies [7]–[9]. The graph-style policy in our probe follows morphology-general RL from locomotion [10]–[12]. RL has been used for the docking maneuver of two vehicles [13]. To our knowledge, as of a literature search in October 2026, no work uses RL to control the already-docked rigid structure.

Our classical baselines build on adaptive control, in which INDI handles model error through measured angular acceleration [14]–[16]. Recursive least-squares estimation of control effectiveness has been used on single quadrotors [17], [18]. We include an INDI baseline and build the estimator on the same idea of regression on thrust.

III. System and Tasks

The simulator integrates a single rigid body for all modules in a batch. The physical step is 0.002 s and the controller runs at 100 Hz; each motor follows a first-order lag with time constant 0.07 s; gyroscope and accelerometer noise have standard deviations 0.01 rad/s and 0.05 m/s^2. Docked modules form one body with no contact or joint dynamics; a crash is declared when height falls below zero, position error exceeds 3 m, or tilt exceeds 100°. Episodes last 5 s. Module mass, inertia, and spacing come from a CAD model of the module (`cad/`). Configuration *line-N* is N modules in a row; we use $N = 6, 7, 8$; *grid10* is 10 modules in a 5×2 grid (5 along x, 2 along y), used only in the mid-flight event study. Fig. 1 shows the top view of line-6, line-7 and line-8 and the payload attach points of E1 and E3.

We evaluate three tasks with one score. `T2` (the tracking task) tracks a set point that changes every 1.5 s (position within ± 1 m, yaw within $\pm 90°$). `POS_STEP` (the position step) is a 1.5 m position step and `ATT_STEP` a 0.8–1.2 rad attitude step; *HOVER* is `POS_STEP` with a zero step, so the payload event happens during hover (event study only). The score of an episode is success $\times \max(0,\, 1 - e_{\mathrm{rms}}/e_{\mathrm{ref}})$, with $e_{\mathrm{ref}} = 1.0$ rad for `ATT_STEP` and 1.5 m for `POS_STEP` and `T2`. A score of 1 is perfect and 0 is a crash or no correction. A recovery task, used only for RL training, starts from a random attitude and returns to hover. Each cell uses 100 episodes.

The payload is a point mass attached at a module center or at a fixed offset; its mass is a fraction of the total mass. The conditions used here are:

- **E1**: a 15% payload on the module with largest x, no offset, present from $t = 0$.
- **E3**: E1 plus an offset of $(+0.0333, -0.0333, -0.0215)$ m, a corner of the training range of the offset (so not out-of-distribution).
- **Event**: the payload is attached or dropped mid-flight, at a module center or off-center.
- **Nominal**: no payload.

A 15% payload is feasible for hover at a thrust-to-weight ratio of 1.36; a 30% payload is infeasible at 1.12 on all six configurations tested, so payload studies stay at 10–15%; residual-RL training randomizes the payload up to 20%.

This comparison favors the classical controllers. They read true position, velocity, attitude, and angular rate, and the leader knows the nominal shape; the RL actor sees true position, velocity, and attitude but a noisy gyroscope and accelerometer, and only k-hop shape information. A variant of the classical stack with noisy angular rate (called A3 in the pre-registration) was specified but not run.

IV. Classical Controller Ladder

All classical controllers share one outer loop: a position PD law produces a desired acceleration, from which a geometric attitude law produces an angular-acceleration command; the gains are normalized by mass and inertia, so one set serves all configurations. They differ in the allocation layer and in what they know about the payload.

- **B1**: the leader knows the nominal shape and allocates by pseudoinverse and clipping.
- **B1-QP**: the same wrench, allocated by bounded least squares; under saturation roll and pitch have priority over thrust, and thrust over yaw.
- **yawlim**: the yaw reference follows the target at a bounded rate $\mathrm{rate}_c \sqrt{\alpha_{\mathrm{yaw}}}$, where α_{yaw} is the nominal yaw angular-acceleration authority of the configuration. We use $\mathrm{rate}_c = 0.556$, selected from 20 settings on training configurations.
- **estim**: payload estimation by recursive least squares with forgetting. Total mass is estimated by regressing accelerometer specific force on summed thrust; the center-of-mass offset is estimated by regressing the torque residual on summed thrust. The payload is then treated as a point mass to rebuild the center of mass, inertia, and allocation matrix. The estimator uses only the noisy gyroscope and accelerometer and the controller's own motor commands.
- **estim-reset**: **estim** plus a CUSUM change detector that resets the regression after a mid-flight event.
- **B3 INDI** and **B3+adapt**: incremental dynamic inversion, without and with a per-axis gain adaptation.
- **informed** controllers (called the informed reference in the text) are given the true payload model and allocate with it. **informed-offset** also knows the offset; **informed-event** also knows the event.

The main configuration is **B1-QP-estim-yawlim** (**estim-yawlim** below). Its optional snap step projects the center-of-mass estimate to the nearest module center; the main configuration has snap off.

The informed controllers are references, not upper bounds. Their angular-acceleration limits remain nominal, and in conditions with thrust-coefficient and docking errors they do not know those errors. A controller outside this family could score higher.

The classical side had 20 settings each for the estimator, yawlim, and reset detector (60 in total). The estimator and detector settings were scored on 32 validation episodes per setting; yawlim was tuned on training configurations. The estimator settings were selected with snap on and a payload at

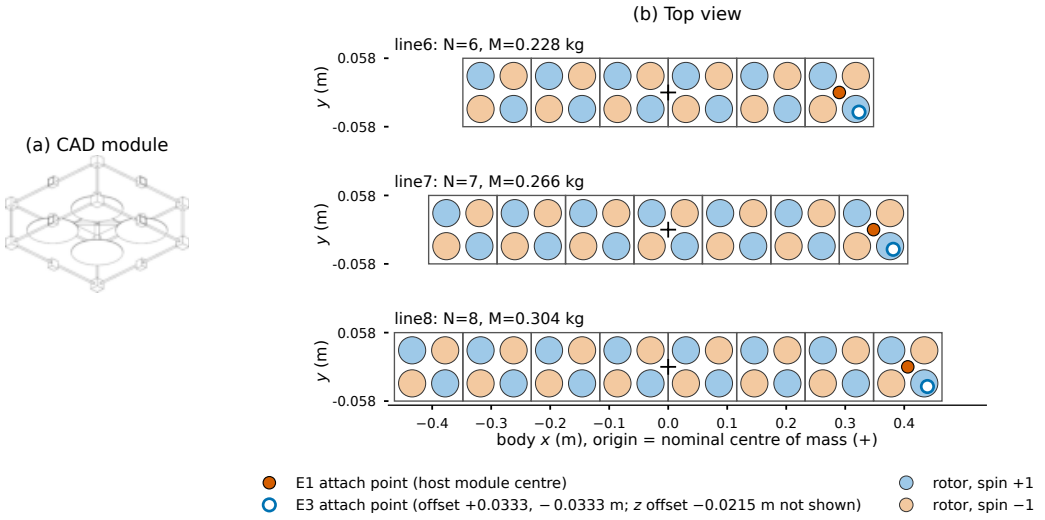


Fig. 1. System overview. (a) CAD model of one module. (b) Top view of line6, line7 and line8 in the body frame (x forward, y left), module pitch 0.116 m, rotor radius 0.023 m, each module with four rotors at $(\pm 0.028, \pm 0.028)$ m from its center; the origin (+) is the nominal center of mass (without payload). Filled marker: E1 payload attach point at the center of the module with largest x. Open marker: E3 attach point, offset $(+0.0333, -0.0333)$ m in the x–y plane; the z offset of -0.0215 m is not visible in this view.

a module center, and then run with snap off and no retuning. All confirmatory results use the same settings, without retuning.

V. Reinforcement-Learning Probe

The probe uses a PPO [19] written from scratch (`ppo.py`); Appendix A compares it with CleanRL [20]. The policy is a graph network over modules with $k = 2$ hops of neighbor information and a history of $H = 4$ steps; training uses 512 environments per (configuration, task) pair (1024 for the two-task runs here) and hidden width 64. The actor outputs a normalized command per motor. It cannot observe the payload. The critic receives four privileged payload fields during training only. The base recipe trains on line6 with a recovery task (`T1`) and the `T2` task for 1.5×10^8 steps from training seed 7.

The pre-registration allowed at most two recipe changes after the base recipe. We tried (i) a penalty on action means outside the bounds (mean-bound, 4×10^7 steps), and (ii) one sample per team with the same penalty (1.5×10^8 steps). If line6 did not meet the pre-registered precondition after the budget, the main RL experiment would not be opened (prereg. rule 7.3(b)).

Because the main experiment was not opened, we trained a residual policy: $u = \mathrm{clip}(u_{\text{ESTIM-}yawlim} + 0.2\,\delta)$. It trains on line6 `T2` and `POS_STEP` with payloads up to 20% and random offsets, stopped by a 120-minute cap after 3.27×10^7 steps (1000 of 1831 planned iterations). It was evaluated on validation and exploration seeds only and never on confirmatory seeds.

Most recipes have a single development seed; seed 8 was added for the mean-bound and residual recipes after the freeze. All RL conclusions below are exploratory.

VI. Protocol

Pre-registration v1.0 was frozen on 2026-11-01 under the tag `prereg-v1.0`, before any RL policy was evaluated on confirmatory seeds; in the end none was. The tag was first placed on an unfinished commit and moved a few minutes later, before any confirmatory result existed. Three seed sets, with environment seeds disjoint by construction, are used as in Table I. Training environment seeds (seed $\times\, 1000 + i$) lie outside all three sets. The formal training seeds 101–112 were never used; the development runs use seeds 7 and 8.

TABLE I
Seed sets (environment-seed offsets).

Set	Offset	Use
Exploration	0	classical diagnostics; early RL numbers
Validation	100	tuning of the estimator; RL evaluation reported here, except the four-configuration run (offset 0)
Confirmatory	900000	classical recomputation after the freeze

Two choices were made after the classical exploration results were known, and before any RL evaluation. First, snap off as the main setting was chosen after diagnostics showed that snap on fails for off-center payloads. Second, the E3 cell was chosen after the classical gap decomposition. In addition, the main E3 task was switched from `T2` to `POS_STEP` by a rule written in advance (largest mean relative gap on exploration seeds); the switch was made before any RL evaluation on E3. Confirmatory E3 scores had already been computed before the freeze and entered in the pre-registration, so E3 is not a blind confirmation.

None of the pre-registered hypothesis tests was run. The pre-registration specified equivalence tests on E1 (two one-sided t intervals over training seeds) and one-sided superiority tests

on E3 POS_STEP with Holm correction [21]. They compare the trained RL policy with estim-yawlim over formal training seeds, and the main RL experiment that would supply those seeds was not opened (rule 7.3(b)). The formal precondition evaluation (training seed 101) was also never run; the decision rested on the development runs. We therefore wrote the RL outcome as "did not meet the pre-registered precondition for opening the main RL experiment within the recipe budget," not as a failed test. No RL-versus-classical hypothesis test is reported.

After the freeze we recomputed the classical results on confirmatory seeds: 384 cells, no retuning. All directions agree with exploration seeds. A mechanical rule flagged 9 comparison rows as inconsistent; in all nine the exploration–confirmatory difference is within ± 0.015 and the confirmatory paired difference is at most 0.006, so no conclusion flips (rows listed in results/p2/classical_confirm.md). Confirmatory T2 scores run about 0.02–0.05 above exploration scores. Intervals are 95% percentile bootstrap intervals over episodes (2000 resamples) [22]; paired differences are bootstrapped per episode. They are conditional on the fixed set of 100 episodes and do not cover the choice of that set. One erratum: the pre-registration text said that 6 of 12 cells of the multi-configuration RL run were below 0.90 of B1; the correct count is 4. A second slip, a swapped snap-on/snap-off order in one table, was corrected on 2026-10-31 and does not change a conclusion.

VII. Results

A. Payload is the costliest of the three model errors compared (exploration seeds only)

Over five configurations and two tasks, the best nominal classical baseline (B1-QP) scores 0.497. With a 15% payload, the best classical baseline under payload (INDI) loses 14.4% of its own nominal score, and B1-QP loses 90.4% of its own. A $\pm 20\%$ thrust-coefficient error costs the best baseline 1.7%, and ± 5 mm, $\pm 2^\circ$ docking error costs 0.3% (both B1-QP, each relative to its own nominal score). These numbers use exploration seeds and were not recomputed on confirmatory seeds. We therefore treat payload as the main error in the rest of the paper.

B. With a centered payload, estim-yawlim stays close to the informed reference (confirmatory)

Table II and Fig. 2(a) give the E1 results. Without payload knowledge, B1-QP-yawlim collapses on T2: 0.021, 0.011, 0.009 on line6/7/8, with success 11%, 6%, 3%. The INDI baseline reaches 0.224, 0.181, 0.155 on T2. estim-yawlim scores 0.340, 0.310, 0.283, and the informed reference scores 0.341, 0.311, 0.289. The paired difference informed minus estim on T2 is $+0.001$, $+0.001$, $+0.006$, and all three 95% intervals contain 0. On POS_STEP the paired differences are $+0.002$, $+0.003$, $+0.007$, and the intervals exclude 0 but are small. Within the point-mass payload model, estim-yawlim therefore reaches the informed reference to within 0.007 on E1; no equivalence test was run, and the widest interval (line8 T2) has upper bound $+0.019$, about 7% of the reference score. Relative to its own no-payload score (not listed in Table II), estim-yawlim keeps 0.815, 0.748, 0.689 on T2 and 0.987, 0.978, 0.953 on POS_STEP. The loss on T2 grows with the number of modules. The reference shows the same pattern (0.341, 0.311, 0.289 against a nominal 0.417, 0.415, 0.411). We read this as a physical limit and not an estimation limit (our interpretation; the pre-registration says the same for exploration snap-on numbers).

yawlim itself matters on the nominal task: on line8 T2, B1-QP scores 0.328 (success 87%) and B1-QP-yawlim 0.411 (100%).

C. With an off-center payload, a gap remains in the position step (confirmatory)

With an off-center payload, B1-QP and B1-QP-yawlim score 0 on every cell. On T2 estim-yawlim and the informed reference differ by at most 0.005 in score, and all three intervals contain 0. On POS_STEP a gap remains: informed minus estim is $+0.047$, $+0.061$, $+0.108$ for line6/7/8, with 95% intervals that exclude 0. Relative to the informed score (0.595, 0.559, 0.494) this is 8%, 11%, 22%; computed from the table values it is 7.9%, 10.9%, 21.9%. We use this denominator throughout. The same POS_STEP data give informed/estim ratios of 1.086, 1.123, 1.281.

On exploration seeds a factorial analysis helps to locate the gap (not recomputed on confirmatory seeds). Here the gap is the percent shortfall relative to the informed reference, pooled over the six E3 cells (three configurations, two tasks). Starting the payload at $t = 0$ instead of attaching it later reduces the gap by 6.7 pp, and a random instead of farthest mount reduces it by 6.0 pp; the estimator version reduces it by 0 pp. The source marks the mechanism as inference, and we have not isolated it.

With an off-center payload, the estim-yawlim stack with snap on scored 0.000 on line6 and line8 T2, against 0.197 and 0.131 with snap off (success 0% versus 74% and 58%; exploration seeds). With the payload at a module center the two settings differ by at most 0.002. The estimator was tuned and evaluated only with centered payloads, so the failure was not visible during tuning.

D. After mid-flight payload events, the reset estimator recovers the informed reference on average (confirmatory)

We compare estim-yawlim with and without the change-point reset against the informed reference that knows the event, averaged over four configurations (line6, line7, line8, grid10) and three tasks. With the reset, the estimator reaches 0.98–1.00 of the informed reference's score in all twelve task-by-condition rows. Without the reset, it reaches 0.89–0.98 for centered events and 0.67–0.89 for off-center events. Within the point-mass model, the reset estimator thus recovers the informed reference on average. Per-configuration rows are in results/p2/classical_confirm.md (the EV section); the aggregate is computed in paper/figs/rqd_aggregate.csv.

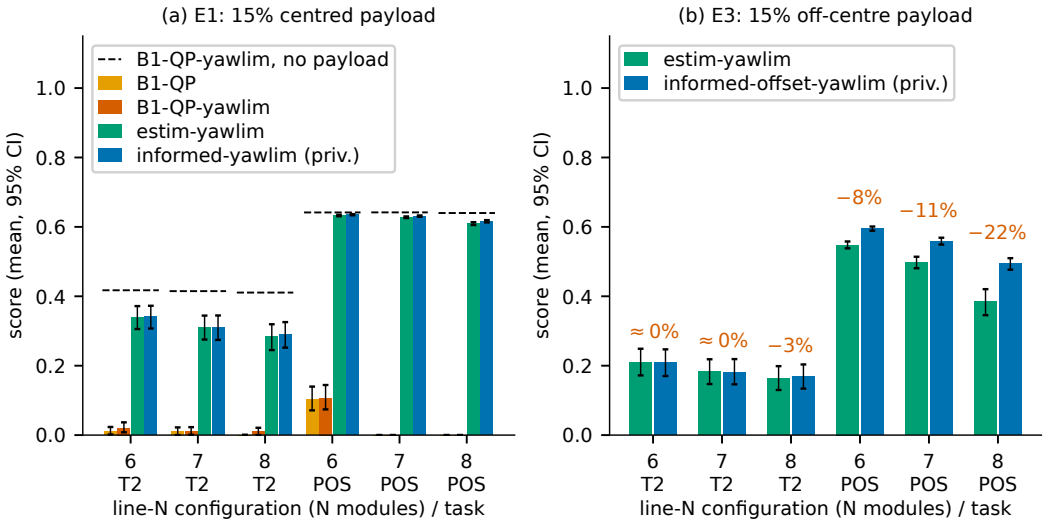


Fig. 2. estim-yawlim tracks the informed reference closely in all cells except the off-center position step, where the gap grows with module count. Classical results on confirmatory seeds (seed offset 900000, 100 episodes per cell). Informed is informed-yawlim in (a) and informed-offset-yawlim in (b). (a) E1, centered 15% payload. (b) E3, off-center 15% payload. Percentages in (b): gap of estim-yawlim relative to informed-offset-yawlim, (inf−est)/inf, from the plotted means (≈0%: below 0.5%). Dashed: B1-QP-yawlim without payload (same seeds). Error bars: 95% bootstrap CI over episodes. `T2` differences between estim-yawlim and informed lie within the 95% CIs. POS = `POS_STEP`.

TABLE II

WITH A CENTERED PAYLOAD ESTIM-YAWLIM STAYS CLOSE TO THE INFORMED REFERENCE; WITH AN OFF-CENTER PAYLOAD IT FALLS SHORT ON THE POSITION STEP, MORE SO WITH MORE MODULES. CLASSICAL RESULTS ON CONFIRMATORY SEEDS (OFFSET 900000, 100 EPISODES PER CELL). SCORES ARE MEANS; "INFORMED" IS THE INFORMED REFERENCE (INFORMED-YAWLIM FOR E1, INFORMED-OFFSET-YAWLIM FOR E3), NOT AN UPPER BOUND. THE LAST COLUMN IS INFORMED − ESTIM (PAIRED, 95% BOOTSTRAP CI). GAP IS (INFORMED − ESTIM)/INFORMED FOR E3 `POS_STEP`. THE "NO PAYLOAD" COLUMN IS B1-QP-YAWLIM WITHOUT PAYLOAD; THE B1-QP-YAWLIM COLUMN IS WITH THE PAYLOAD OF THE ROW'S CONDITION.

Cond.	Task, config	No payload	B1-QP-yawlim	estim-yawlim	informed	informed − estim [95% CI]	Gap
E1	`T2`, line6	0.417	0.021	0.340	0.341	+0.001 [−0.001, +0.003]	–
E1	`T2`, line7	0.415	0.011	0.310	0.311	+0.001 [−0.006, +0.005]	–
E1	`T2`, line8	0.411	0.009	0.283	0.289	+0.006 [−0.005, +0.019]	–
E1	`POS_STEP`, line6	0.642	0.107	0.633	0.635	+0.002 [+0.002, +0.003]	–
E1	`POS_STEP`, line7	0.642	0.000	0.628	0.631	+0.003 [+0.003, +0.004]	–
E1	`POS_STEP`, line8	0.640	0.000	0.610	0.616	+0.007 [+0.006, +0.008]	–
E3	`T2`, line6	0.417	0.000	0.210	0.209	−0.001 [−0.008, +0.006]	–
E3	`T2`, line7	0.415	0.000	0.182	0.181	−0.001 [−0.010, +0.007]	–
E3	`T2`, line8	0.411	0.000	0.163	0.168	+0.005 [−0.007, +0.016]	–
E3	`POS_STEP`, line6	0.642	0.000	0.548	0.595	+0.047 [+0.041, +0.052]	8%
E3	`POS_STEP`, line7	0.642	0.000	0.498	0.559	+0.061 [+0.052, +0.070]	11%
E3	`POS_STEP`, line8	0.640	0.000	0.385	0.494	+0.108 [+0.084, +0.134]	22%

E. RL neither met the precondition nor closed the gap (validation seeds, exploratory)

Table III and Fig. 3 give the RL results.

All RL numbers use validation seeds, 100 episodes per cell, and the final checkpoint, except the four-configuration run, which used exploration seeds (offset 0); all are exploratory. RL scores are not comparable with the confirmatory numbers of Table II because the seed sets differ.

From scratch on line6 without payload, the base recipe reaches 0.61, 0.70, 0.79 of B1 on `ATT_STEP`, `POS_STEP`, `T2`. Against the stronger of B1 and B1-QP, written max(B1, B1-QP), the same run gives 0.606, 0.701, 0.745. Its training success rate fell during training (`T2` from 0.61 to 0.45). The RL policy saturated its actions in 0.490 of steps on line6 `T2`, against 0.149 for B1 and 0.170 for B1-QP. The recorded failure mode is tilt flip. The mean-bound penalty reduced saturation to 0.157 but did not improve all tasks: ratios of 0.491, 0.892, 0.929 for seed 7 and 0.634, 0.707, 0.661 for seed 8. The mean-bound run also used a different learning-rate schedule than the base run, so the comparison is not controlled. The team-sample recipe gives 0.759, 0.405, 0.626, and the recipe budget is used up. Every recipe left at least one no-payload task below $0.90\times$B1 (Table III), so we judged the precondition for the main experiment unmet.

As a pipeline check, a single-module policy trained on the same code reaches 1.046, 1.054, 1.099 of max(B1, B1-

TABLE III

EVERY FROM-SCRATCH RECIPE FELL BELOW 90% OF B1 ON AT LEAST ONE NO-PAYLOAD TASK, AND THE RESIDUAL POLICY DID NOT IMPROVE ON ESTIM-YAWLIM. RL PROBE ON VALIDATION SEEDS (OFFSET 100, 100 EPISODES PER CELL; EXPLORATORY; FINAL CHECKPOINT). (A) FROM SCRATCH ON LINE6 WITHOUT PAYLOAD, AS A RATIO OF THE RL SCORE TO THE DENOMINATOR NAMED IN THE LAST COLUMN. "B1" MEANS THE SCORE OF B1; "MAX" MEANS MAX(B1, B1-QP). (B) RESIDUAL RL ON LINE6 E3, ABSOLUTE SCORES; THE RESIDUAL IS ADDED TO ESTIM-YAWLIM, THE CLASSICAL ESTIMATOR STACK.

(a) From scratch, line6 nominal: RL score / denominator					
Recipe	Train seed	ATT_STEP	POS_STEP	T2	Denominator
Base	7	0.61	0.70	0.79	B1
Base	7	0.606	0.701	0.745	max
Mean-bound	7	0.491	0.892	0.929	B1
Mean-bound	8	0.634	0.707	0.661	B1
Team-sample	7	0.759	0.405	0.626	B1
Pipeline check, single module (not line6; one policy, seed 0): ratio to max					
Single-module mix	0	1.046	1.054	1.099	max
(b) Residual RL, line6 E3: absolute score					
		ATT_STEP	POS_STEP	T2	
Residual	7	0.353	0.554	0.225	–
Residual	8	0.353	0.555	0.224	–
estim-yawlim	–	0.354	0.565	0.239	–
informed reference	–	0.358	0.604	0.245	–

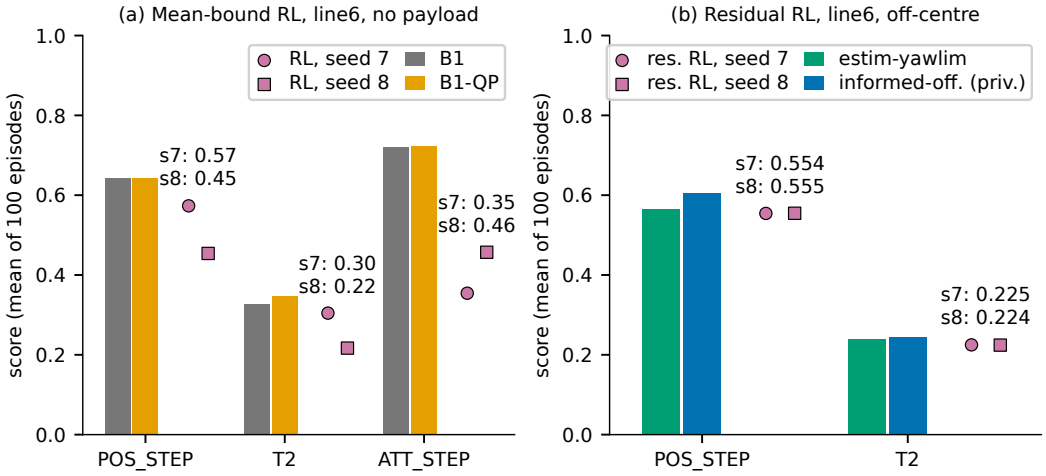


Fig. 3. Mean-bound RL stays below B1 on every no-payload task, and the residual policy stays slightly below estim-yawlim under an off-center payload. RL probe on validation seeds (seed offset 100, 100 episodes per cell); exploratory. (a) Mean-bound RL on line6 without payload; (b) residual RL on line6 with off-center payload (E3). Circle = RL training seed 7, square = RL training seed 8; label = seed 7/seed 8. Bars: classical controllers evaluated on the same validation episodes. Error bars: none (the RL/validation JSON files store only per-cell means, no per-episode scores or CI). Not comparable with Fig. 2 (different seed group).

QP) with 100% success. A policy trained jointly on four configurations plateaued, with 4 of 12 cells below 0.90 of B1 (exploration seeds). We report this as a sanity check only; the multi-configuration problem is unsolved here.

On line6 E3 POS_STEP, the residual policy scores 0.554 (seed 7) and 0.555 (seed 8), against 0.565 for estim-yawlim and 0.604 for the informed reference. The paired differences against estim-yawlim are -0.010 $[-0.013, -0.008]$ and -0.010 $[-0.011, -0.009]$. On E3 T2 it scores 0.225 and 0.224, against 0.239 and 0.245. The two seeds differ by at most 0.002 on every cell, and each seed was trained once. The residual did not close the gap: it is slightly below estim-yawlim on both seeds, and two seeds that agree to within 0.002 are consistent with a correction close to zero (its magnitude was not measured). Nominal-condition T2 for the residual is 1.27 of B1, but the evaluation did not list nominal estim-yawlim, so we do not credit the residual for it.

VIII. LIMITATIONS

Each item names the conclusion it limits and what the reader should not infer.

- *Two development seeds for RL.* Most recipes were trained with one development seed, and only the mean-bound and residual recipes with a second (seed 8). We consider one

or two seeds too few for a conclusion about RL, and the two mean-bound seeds disagree strongly (`T2` ratio 0.929 and 0.661). The RL results show only that these one- or two-seed runs, these recipes, and this compute budget did not close the gap; they do not show that RL cannot. The residual runs also hit the time cap, and the mean-bound recipe differs from the base in learning-rate schedule.

- *Pre-registration frozen after classical exploration.* Snap off and the E3 cell were chosen after the classical results were seen, and the confirmatory E3 scores were seen before the freeze (Section VI). The E3 result is therefore not a blind confirmation, and E3 was chosen knowing where the exploration gap appeared.
- *Simulation and a point-mass payload.* There is no hardware, and the estimator and the environment share the point-mass payload model, so "recovers" holds only within that model. Off-center payloads and snap off test the circularity only in part, and the results do not transfer to hardware or to non-point-mass, flexible, or jointed payloads.
- *Estimator tuned on centered payloads only.* The estimator was tuned only with centered payloads, and we did not test retuning on off-center payloads, so the off-center result applies to the estimator as tuned.
- *Tuning and information favor the classical side.* The classical stack reads true state and the full nominal shape, while the RL actor sees a noisy gyroscope and accelerometer and only k-hop shape information (Section III); the noisy-rate variant A3 was not run. The classical gains and settings were tuned on training configurations and validation seeds, and the RL evaluation reuses those validation seeds. The classical budget (60 settings) and the RL budget (at most 7 trainings) are not commensurable, so we claim no parity and no RL-versus-classical ranking.
- *Fixed evaluation set.* Intervals cover episode sampling only, not the choice of the 100 episodes.
- *E3 is a corner of the training offset range.* It is not a held-out offset, so the off-center result says nothing about offsets outside the training range.
- *One centralized brain* with ideal communication, so the results do not cover delayed or lossy links between modules.

IX. Conclusion

In simulation, an unknown payload is the costliest of the three model errors we compared (exploration seeds; the magnitudes are not on a common scale). On confirmatory seeds and within the point-mass payload model, the estimator stack (QP allocation, a yaw rate limit, and online mass and center-of-mass estimation) is within 0.007 score of the informed reference for centered payloads. For an off-center payload, a gap of 8/11/22% remains in the position step. Within the tested recipes and budget, a from-scratch PPO did not reach the classical baselines on line6 without payload, and an exploratory residual PPO did not close the off-center gap in two training seeds. No test of RL against classical control was run, so the RL result is a negative probe and not a verdict. The protocol lesson is that freezing, recomputing on fresh seeds, and publishing the erratum are cheap in a small study and make every number above traceable. Future work that the repository supports: a fix for multi-configuration RL training, offsets held out of the training range, and hardware.

Disclosure of AI assistance

The code, experiment scripts and a draft of this paper were written by an AI assistant (Claude, Anthropic) under the author's direction. The author reviewed the work and decided the research direction and all trade-offs.

Appendix

We checked `ppo.py` against CleanRL [20] on Hopper for 10^6 steps. The acceptance rule was changed after seeing results. The original rule, that all three of our seeds fall inside the CleanRL range, was not met (our lowest seed 1860 against a CleanRL minimum of 1940). We replaced it with: the 95% bootstrap CI of the mean difference lies within $\pm 15\%$ of the CleanRL mean. At 8 seeds the CI was $[-14.2\%, +15.1\%]$ and narrowly missed; we then extended to 12 seeds as a declared sequential extension. At 12 seeds the means are 2275.3 (ours) and 2268.1 (CleanRL), a difference of $+0.3\%$ with CI $[-10.8\%, +11.6\%]$, which passes. Because the rule was applied sequentially, the nominal α is slightly inflated. The Hopper seeds are training seeds and are not environment seeds of the three sets in Table I.

References

[1] D. Saldaña, B. Gabrich, G. Li, M. Yim, and V. Kumar, "ModQuad: The flying modular structure that self-assembles in midair," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 691–698.

[2] J. Xu, D. S. D'Antonio, and D. Saldaña, "Modular multirotors: From quadrotors to fully-actuated aerial vehicles," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 19 328–19 339, 2025, arXiv:2202.00788 (H-ModQuad).

[3] B. Mu and P. Chirarattananon, "Universal flying objects: Modular multirotor system for flight of rigid objects," *IEEE Transactions on Robotics*, vol. 36, no. 2, pp. 458–471, 2020, arXiv:1911.03615.

[4] R. Huang, Z. Cai, S. Tang, P. Wei, L. Li, X. Chen, W. Cao, Z. Zhang, and L. Zhao, "MARS-Dragonfly: Agile and robust flight control of modular aerial robot systems," 2026, arXiv:2604.05499.

[5] R. Huang, Z. Gao, S. Tang, J. Zhang, L. He, Z. Zhang, and L. Zhao, "TransforMARS: Fault-tolerant self-reconfiguration for arbitrarily shaped modular aerial robot systems," 2025, arXiv:2509.14025; arXiv comment: ICRA 2026.

[6] O. Konstantaropoulos, W. Rehberg, M. Kulkarni, and K. Alexis, "Embodiment-conditioned generalist control for multirotor aerial robots," 2026, arXiv:2606.10857.

[7] V. Vaidya and J. Keshavan, "Dynamics-invariant quadrotor control using scale-aware deep reinforcement learning," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, arXiv:2503.09622.

[8] J. Eschmann, D. Albani, and G. Loianno, "RAPTOR: A foundation policy for quadrotor control," 2025, arXiv:2509.11481.

[9] W. Rehberg, M. Kulkarni, P. Weiss, and K. Alexis, "Efficient knowledge transfer for jump-starting control policy learning of multirotors through physics-aware neural architectures," *IEEE Robotics and Automation Letters*, 2026, accepted; volume and pages not verified. arXiv:2602.15533.

[10] W. Huang, I. Mordatch, and D. Pathak, "One policy to control them all: Shared modular policies for agent-agnostic control," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, arXiv:2007.04976.

[11] L. Engwegen, M. Weltevrede, C. Horsch, D. Brinks, and W. Böhmer, "Shared modular recurrence in contextual MDPs for universal morphology control," 2025, arXiv:2506.08630.
[12] Y. Luo, M. Yao, and X. Xiao, "GCNT: Graph-based transformer policies for morphology-agnostic reinforcement learning," 2025, arXiv:2505.15211.
[13] Y. Feng, T. Yang, and Y. Yu, "Enhancing UAV aerial docking: A hybrid approach combining offline and online reinforcement learning," *Drones*, vol. 8, no. 5, p. 168, 2024.
[14] E. J. J. Smeur, Q. Chu, and G. C. H. E. de Croon, "Adaptive incremental nonlinear dynamic inversion for attitude control of micro air vehicles," *Journal of Guidance, Control, and Dynamics*, vol. 39, no. 3, pp. 450–461, 2016.
[15] E. J. J. Smeur, G. C. H. E. de Croon, and Q. Chu, "Cascaded incremental nonlinear dynamic inversion for MAV disturbance rejection," *Control Engineering Practice*, vol. 73, pp. 79–90, 2018, arXiv:1701.07254.
[16] S. Sun, X. Wang, Q. Chu, and C. de Visser, "Incremental nonlinear fault-tolerant control of a quadrotor with complete loss of two opposing rotors," *IEEE Transactions on Robotics*, 2020, volume and pages not verified. arXiv:2002.07837.
[17] T. M. Blaha, E. J. J. Smeur, and B. D. W. Remes, "Control of unknown quadrotors from a single throw," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 10 350–10 355, arXiv:2406.11723.
[18] T. M. Blaha, E. J. J. Smeur, B. D. W. Remes, and C. C. de Visser, "Flying a quadrotor with unknown actuators and sensor configuration," 2024, arXiv:2409.01080; submitted to IMAV 2024, publication not confirmed.
[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, arXiv:1707.06347 (no formal venue; arXiv preprint only).
[20] S. Huang, R. F. J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, and J. G. M. Araújo, "CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms," *Journal of Machine Learning Research*, vol. 23, no. 274, pp. 1–18, 2022.
[21] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979, jSTOR 4615733.
[22] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993, iSBN 0-412-04231-2.